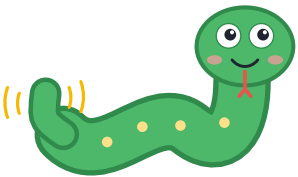
Lecture 10

Part 4: Data Science and AI

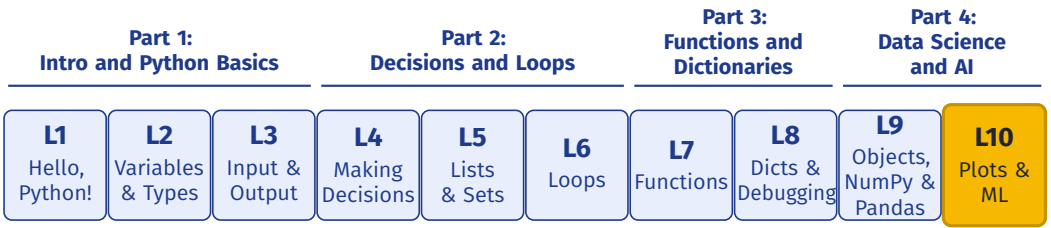
Plots, Machine Learning & Wrap-up

Pictures from data, a first taste of AI, and where to go next



Dror Jacoby
Tel Aviv University

Where we are


Part 1:
Intro and Python Basics
Part 2:
Decisions and Loops
Part 3:
Functions and
Dictionaries
Part 4:
Data Science
and AI
L1
Hello,
Python!
L2
Variables
& Types
L3
Input &
Output
L4
Making
Decisions
L5
Lists
& Sets
L6
Loops
L7
Functions
L8
Dicts &
Debugging
L9
Objects,
NumPy &
Pandas
L10
Plots &
ML
You are here

Which one tells the story faster?

My sleep (hours)

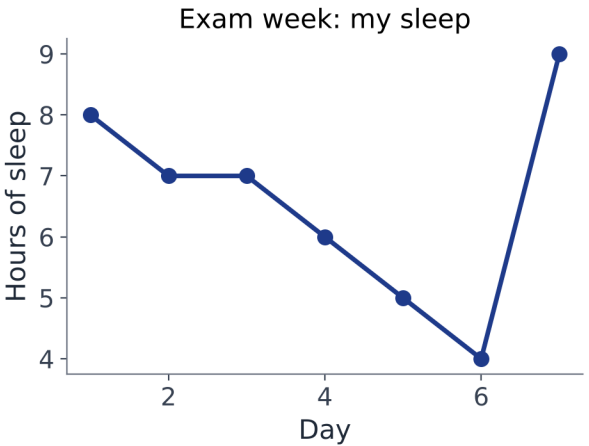
```
8, 7, 7, 6, 5, 4, 9
```

Made-up numbers

Which one tells the story faster?

My sleep (hours)
8, 7, 7, 6, 5, 4, 9

Numbers: you have to think.
Plot: you just **see** it.
Exam on day 6, then sleep!



Made-up numbers

Today's plan

◎ By the end of today you can…

- Draw line, bar, scatter and histogram plots
- Train a tiny machine learning model in 3 steps
- See how pros code: an editor, a debugger, an AI assistant
- Connect all 10 lectures and start your final project



Today's outline

1. **Plots with Matplotlib**
   Line, bar, scatter and histogram, with titles and labels
2. **A first taste of machine learning**
   Regression, classification and clustering with scikit-learn
3. **Code like a pro**
   A demo: a real editor (IDE), a step-by-step debugger, an AI assistant
4. **Wrap-up & your final project**
   What we learned, and the final project



Every topic: examples, **Try it!** and **Predict the output**. At the end: **Bug Zoo**, **AI Corner**, **Recap**.

Why today matters

Why learn this?

- A good chart tells the story faster than any table
- Machine learning = learning patterns from data, not from rules
- Everything comes together in your final project

Where you will see it

- Charts in the news and in reports
- Recommendations, spam filters, face unlock
- Medical imaging and weather forecasts





Topic 1

Plots with Matplotlib

Lecture 10 • Plots & ML



TODAY

1. **Plots with Matplotlib**
2. A first taste of machine learning
3. Code like a pro
4. Wrap-up & your final project

The plotting recipe



- **Matplotlib**: Python's most popular plotting library
- We import its `pyplot` part with the nickname `plt`

Your first line plot

L10_01_plots.ipynb

```
import matplotlib.pyplot as plt

days = [1, 2, 3, 4, 5, 6, 7]
sleep = [8, 7, 7, 6, 5, 4, 9]
plt.plot(days, sleep)
plt.show()
```

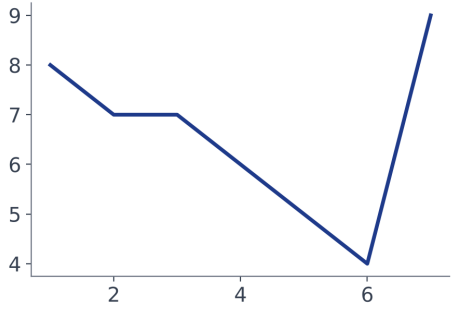
`plt.plot(x, y)`: x goes right, y goes up.

Your first line plot

L10_01_plots.ipynb

```
import matplotlib.pyplot as plt

days = [1, 2, 3, 4, 5, 6, 7]
sleep = [8, 7, 7, 6, 5, 4, 9]
plt.plot(days, sleep)
plt.show()
```



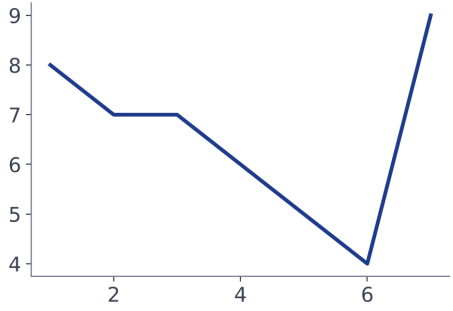
`plt.plot(x, y)`: x goes right, y goes up.

Your first line plot

L10_01_plots.ipynb

```
import matplotlib.pyplot as plt

days = [1, 2, 3, 4, 5, 6, 7]
sleep = [8, 7, 7, 6, 5, 4, 9]
plt.plot(days, sleep)
plt.show()
```



`plt.plot(x, y)`: x goes right, y goes up.

But…what are we looking at?

Always add a title and labels

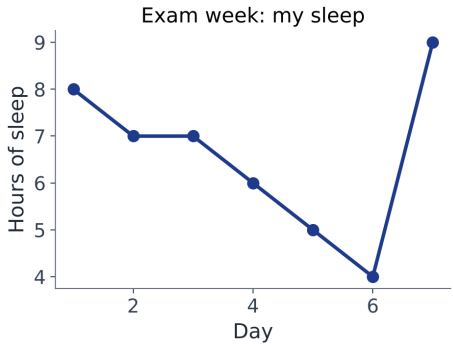
```python
import matplotlib.pyplot as plt

days = [1, 2, 3, 4, 5, 6, 7]
sleep = [8, 7, 7, 6, 5, 4, 9]
plt.plot(days, sleep, marker="o")
plt.title("Exam week: my sleep")
plt.xlabel("Day")
plt.ylabel("Hours of sleep")
plt.show()
```

Always add a title and labels

```python
import matplotlib.pyplot as plt

days = [1, 2, 3, 4, 5, 6, 7]
sleep = [8, 7, 7, 6, 5, 4, 9]
plt.plot(days, sleep, marker="o")
plt.title("Exam week: my sleep")
plt.xlabel("Day")
plt.ylabel("Hours of sleep")
plt.show()
```



`marker="o"` draws a dot on each value.

Two lines on one plot

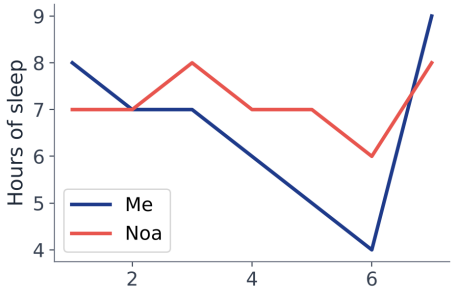
```
import matplotlib.pyplot as plt

days = [1, 2, 3, 4, 5, 6, 7]
me = [8, 7, 7, 6, 5, 4, 9]
noa = [7, 7, 8, 7, 7, 6, 8]
plt.plot(days, me, label="Me")
plt.plot(days, noa, label="Noa")
plt.ylabel("Hours of sleep")
plt.legend()
plt.show()
```

Two lines on one plot

```
import matplotlib.pyplot as plt

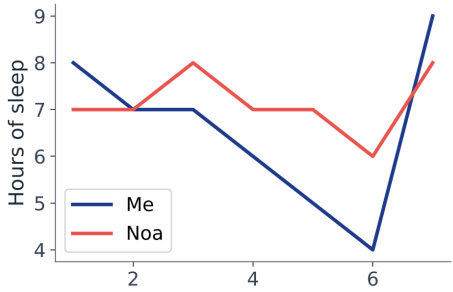
days = [1, 2, 3, 4, 5, 6, 7]
me = [8, 7, 7, 6, 5, 4, 9]
noa = [7, 7, 8, 7, 7, 6, 8]
plt.plot(days, me, label="Me")
plt.plot(days, noa, label="Noa")
plt.ylabel("Hours of sleep")
plt.legend()
plt.show()
```



Two lines on one plot

```
import matplotlib.pyplot as plt

days = [1, 2, 3, 4, 5, 6, 7]
me = [8, 7, 7, 6, 5, 4, 9]
noa = [7, 7, 8, 7, 7, 6, 8]
plt.plot(days, me, label="Me")
plt.plot(days, noa, label="Noa")
plt.ylabel("Hours of sleep")
plt.legend()
plt.show()
```



`label` + `plt.legend()` = a little key.

Noa planned ahead. Be like Noa.

Bar chart: compare groups

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("coffee_orders.csv")
counts = df["drink"].value_counts()
plt.bar(counts.index, counts.values)
plt.title("Orders per drink")
plt.ylabel("Number of orders")
plt.show()
```

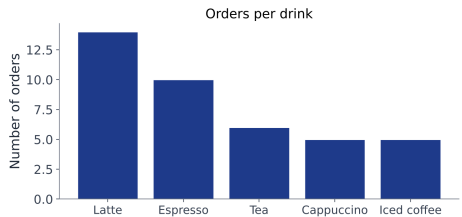
The café data from L09!

`index`: names, `values`: counts

Bar chart: compare groups

```python
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("coffee_orders.csv")
counts = df["drink"].value_counts()
plt.bar(counts.index, counts.values)
plt.title("Orders per drink")
plt.ylabel("Number of orders")
plt.show()
```



The café data from L09!

`index`: names, `values`: counts

Scatter plot: does one thing go with another?

```
import matplotlib.pyplot as plt

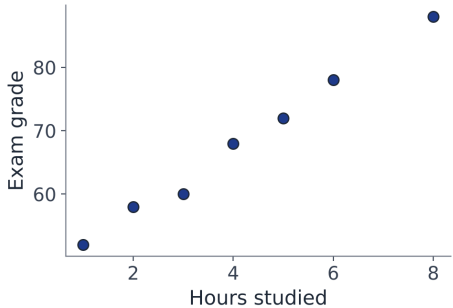
hours = [1, 2, 3, 4, 5, 6, 8]
grades = [52, 58, 60, 68, 72, 78, 88]
plt.scatter(hours, grades)
plt.xlabel("Hours studied")
plt.ylabel("Exam grade")
plt.show()
```

One dot per student. (Made-up data.)

Scatter plot: does one thing go with another?

```python
import matplotlib.pyplot as plt

hours = [1, 2, 3, 4, 5, 6, 8]
grades = [52, 58, 60, 68, 72, 78, 88]
plt.scatter(hours, grades)
plt.xlabel("Hours studied")
plt.ylabel("Exam grade")
plt.show()
```



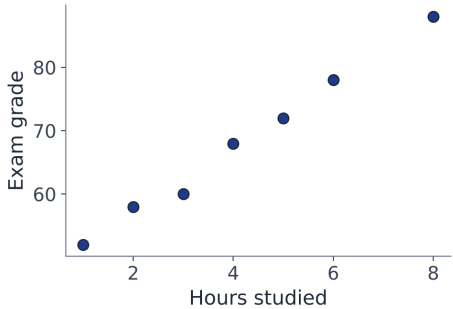
One dot per student. (Made-up data.)

Scatter plot: does one thing go with another?

```python
import matplotlib.pyplot as plt

hours = [1, 2, 3, 4, 5, 6, 8]
grades = [52, 58, 60, 68, 72, 78, 88]
plt.scatter(hours, grades)
plt.xlabel("Hours studied")
plt.ylabel("Exam grade")
plt.show()
```

One dot per student. (Made-up data.)



More study, higher grade.

Remember these dots: they come back soon!

Histogram: how are values spread?

```
import numpy as np
import matplotlib.pyplot as plt

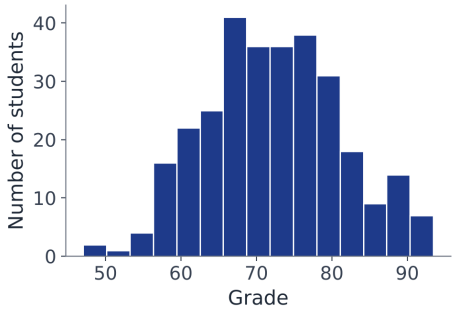
np.random.seed(0)
grades = np.random.normal(72, 9, 300)
plt.hist(grades, bins=15)
plt.xlabel("Grade")
plt.ylabel("Number of students")
plt.show()
```

300 random grades around 72.
Each bar counts grades in a range.

Histogram: how are values spread?

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)
grades = np.random.normal(72, 9, 300)
plt.hist(grades, bins=15)
plt.xlabel("Grade")
plt.ylabel("Number of students")
plt.show()
```



300 random grades around 72.
Each bar counts grades in a range.

Which plot when?

Question	Plot	Command
How does it change over time?	Line	`plt.plot`
Which group is bigger?	Bar	`plt.bar`
Do two things go together?	Scatter	`plt.scatter`
How are values spread?	Histogram	`plt.hist`



Try it!

Try it in Colab notebooks/L10/L10_01_plots.ipynb

1. Plot *your* sleep (or coffee cups) for the last 7 days
2. Add a title and axis labels. Try `marker="s"`
3. Bar chart: how many orders on each `day` of the café data?
4. Histogram: change `bins=15` to 5, then 50. What changes?





Predict the plot

What will you see?

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3], [3, 2, 1])
plt.show()
```

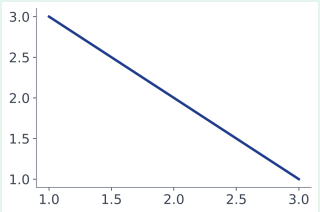
Predict the plot

What will you see?

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3], [3, 2, 1])
plt.show()
```

Answer



A straight line going **down**.
x: 1, 2, 3 y: 3, 2, 1

Topic 2

A first taste of machine learning

Lecture 10 • Plots & ML



TODAY

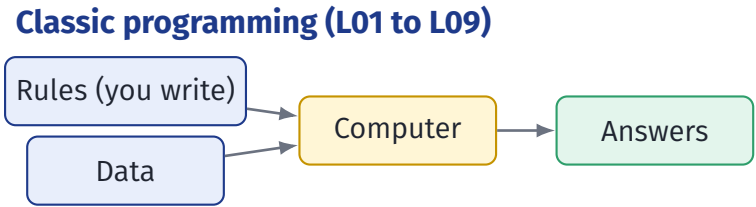
1 Plots with Matplotlib

2 **A first taste of machine learning**

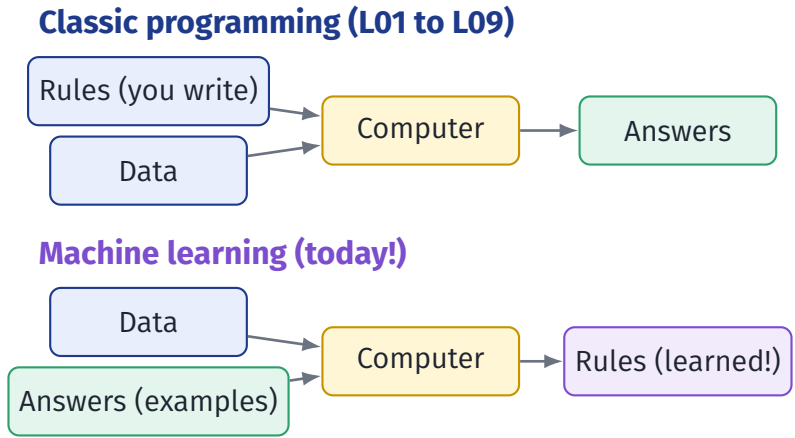
3 Code like a pro

4 Wrap-up & your final project

Remember Lecture 1?



Remember Lecture 1?



Three kinds of questions ML can answer

Regression



Predict a **number**

Study hours → grade

Classification



Pick a **label**

Apple or banana?

Clustering



Find **groups**

Types of café customers

We use **scikit-learn** (`sklearn`), free and pre-installed in Colab.

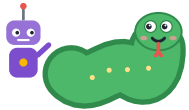
The 3-step recipe (for every model)

1. `model = SomeModel()`
2. `model.fit(examples, answers)`
3. `model.predict(new_examples)`

Like studying

fit = study solved exercises

predict = answer new exam questions



Regression: predict a grade

L10_02_first_ml.ipynb

```
from sklearn.linear_model import LinearRegression

hours = [[1], [2], [3], [4], [5], [6], [8]]
grades = [52, 58, 60, 68, 72, 78, 88]
model = LinearRegression()
model.fit(hours, grades)
print(model.predict([[7]]))
```

Regression: predict a grade

L10_02_first_ml.ipynb

```
from sklearn.linear_model import LinearRegression

hours = [[1], [2], [3], [4], [5], [6], [8]]
grades = [52, 58, 60, 68, 72, 78, 88]
model = LinearRegression()
model.fit(hours, grades)
print(model.predict([[7]]))
```

OUTPUT

```
[82.75409836]
```

Regression: predict a grade

L10_02_first_ml.ipynb

```
from sklearn.linear_model import LinearRegression

hours = [[1], [2], [3], [4], [5], [6], [8]]
grades = [52, 58, 60, 68, 72, 78, 88]
model = LinearRegression()
model.fit(hours, grades)
print(model.predict([[7]]))
```

OUTPUT

```
[82.75409836]
```

- 7 hours of study: about 83
- Double brackets: a table with one column

What did the model learn? A line!

Next cell, same notebook:

```
import matplotlib.pyplot as plt

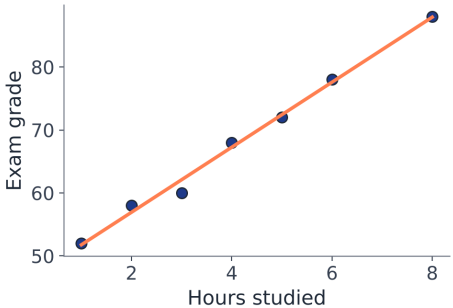
plt.scatter(hours, grades)
plt.plot(hours, model.predict(hours),
         color="coral")
plt.xlabel("Hours studied")
plt.ylabel("Exam grade")
plt.show()
```

What did the model learn? A line!

Next cell, same notebook:

```
import matplotlib.pyplot as plt

plt.scatter(hours, grades)
plt.plot(hours, model.predict(hours),
         color="coral")
plt.xlabel("Hours studied")
plt.ylabel("Exam grade")
plt.show()
```



The “rule” it learned:
the best straight line through the dots.

Careful: models don't know common sense

```
print(model.predict([[30]]))
```



Careful: models don't know common sense

```
print(model.predict([[30]]))
```

OUTPUT

```
[201.52459016]
```



Careful: models don't know common sense

```
print(model.predict([[30]]))
```

OUTPUT

```
[201.52459016]
```

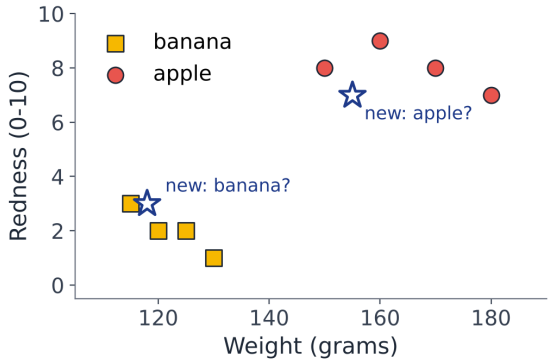


⚠ A grade of 201?!

The model only saw 1 to 8 hours. Far outside its examples, it just continues the line.

Classification: apple or banana?

- Each fruit: **weight** and **redness**
- 8 examples with the right label
- New fruit? Look at its **3 nearest** examples
- Majority vote wins



Made-up fruit. Redness: 0 = yellow, 10 = red.

Classification in code

```
from sklearn.neighbors import KNeighborsClassifier

fruits = [[120, 2], [130, 1], [115, 3], [125, 2],
          [170, 8], [160, 9], [180, 7], [150, 8]]
labels = ["banana", "banana", "banana", "banana",
          "apple", "apple", "apple", "apple"]
model = KNeighborsClassifier(n_neighbors=3)
model.fit(fruits, labels)
print(model.predict([[155, 7], [118, 3]]))
```

Classification in code

```
from sklearn.neighbors import KNeighborsClassifier

fruits = [[120, 2], [130, 1], [115, 3], [125, 2],
          [170, 8], [160, 9], [180, 7], [150, 8]]
labels = ["banana", "banana", "banana", "banana",
          "apple", "apple", "apple", "apple"]
model = KNeighborsClassifier(n_neighbors=3)
model.fit(fruits, labels)
print(model.predict([[155, 7], [118, 3]]))
```

OUTPUT

```
['apple' 'banana']
```

Classification in code

```
from sklearn.neighbors import KNeighborsClassifier

fruits = [[120, 2], [130, 1], [115, 3], [125, 2],
          [170, 8], [160, 9], [180, 7], [150, 8]]
labels = ["banana", "banana", "banana", "banana",
          "apple", "apple", "apple", "apple"]
model = KNeighborsClassifier(n_neighbors=3)
model.fit(fruits, labels)
print(model.predict([[155, 7], [118, 3]]))
```

Same 3 steps: create, `fit`, `predict`.

OUTPUT

```
['apple' 'banana']
```

Clustering: find groups by itself

```
import pandas as pd
from sklearn.cluster import KMeans

df = pd.read_csv("cafe_customers.csv")
model = KMeans(n_clusters=3, random_state=0)
groups = model.fit_predict(df)
print(groups)
```

- 30 café customers: visits, spend
- **No answers given**
- `random_state=0`: same result every run

Clustering: find groups by itself

```python
import pandas as pd
from sklearn.cluster import KMeans

df = pd.read_csv("cafe_customers.csv")
model = KMeans(n_clusters=3, random_state=0)
groups = model.fit_predict(df)
print(groups)
```

- 30 café customers: visits, spend
- **No answers given**
- `random_state=0`: same result every run

OUTPUT

```
[1 1 1 2 2 2 0 2 2 2 0 2 1 0 1 1 0 0 2 0 2 2 1 0 1 0 0 0 1 1]
```

Clustering: find groups by itself

```
import pandas as pd
from sklearn.cluster import KMeans

df = pd.read_csv("cafe_customers.csv")
model = KMeans(n_clusters=3, random_state=0)
groups = model.fit_predict(df)
print(groups)
```

- 30 café customers: visits, spend
- **No answers given**
- `random_state=0`: same result every run

OUTPUT

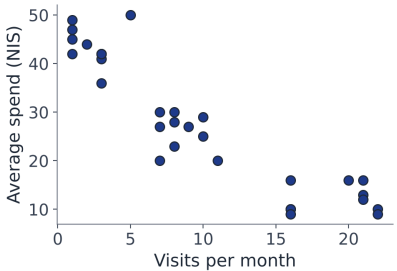
```
[1 1 1 2 2 2 0 2 2 2 0 2 1 0 1 1 0 0 2 0 2 2 1 0 1 0 0 0 1 1]
```

A group number (0, 1 or 2) for every customer.

Let’s see the groups

```
plt.scatter(df["visits"], df["spend"], c=groups)
```

`c=groups`: color each dot by its group (plus labels and `plt.show()`, as before)

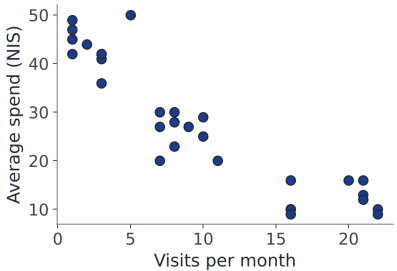


Before: just dots

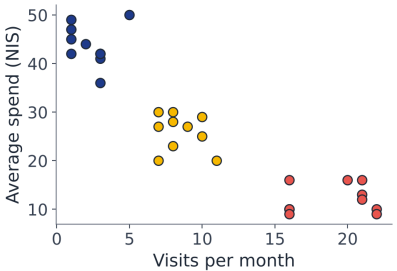
Let's see the groups

```
plt.scatter(df["visits"], df["spend"], c=groups)
```

`c=groups`: color each dot by its group (plus labels and `plt.show()`, as before)



Before: just dots



After: 3 types of customers

Machine learning around you

Kind	In real life
Regression	Delivery time, apartment prices
Classification	Spam or not spam, “cat” photos
Clustering	Customer types, similar songs





Chatbots? A huge model that predicts the next word (L01, L08).

Try it!

Try it in Colab notebooks/L10/L10_02_first_ml.ipynb

1. Predict the grade for 0 hours and for 10 hours. Believable?
2. Add a fruit `[140, 5]`. What does the model say?
3. Change `n_clusters=3` to 2, then 4. Plot each time

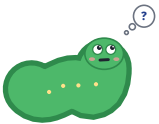


Predict the output

Predict the output!

```
from sklearn.linear_model import LinearRegression

hours = [[1], [2], [3]]
grades = [60, 70, 80]
model = LinearRegression()
model.fit(hours, grades)
print(model.predict([[4]]))
```



Predict the output

Predict the output!

```
from sklearn.linear_model import LinearRegression

hours = [[1], [2], [3]]
grades = [60, 70, 80]
model = LinearRegression()
model.fit(hours, grades)
print(model.predict([[4]]))
```

Answer

`[90.]`

Each hour adds 10 points. 4 hours: 90.



Topic 3

Code like a pro

Lecture 10 • Plots & ML



TODAY

1. Plots with Matplotlib
2. A first taste of machine learning
3. **Code like a pro**
4. Wrap-up & your final project

Code like a pro: an extra, for the curious

- Colab is perfect for learning. Pros also use an **editor** on their own computer
- Today: a quick **demo** of three pro tools: an editor, a step-by-step debugger, an AI assistant
- You already know the hard part: **Python**. The tools are just a new desk



Look for the GOING FURTHER label. A demo, not on the exam. No pressure.

From Colab to a real editor

GOING FURTHER

Colab notebook (`.ipynb`)

Cells, text and plots together. Runs in the browser, on Google's computer. Great for **learning and data stories**.

A Python file (`.py`)

One plain text file, runs top to bottom on **your** computer. This is how apps and websites are built.

From Colab to a real editor

GOING FURTHER

Colab notebook (`.ipynb`)

Cells, text and plots together. Runs in the browser, on Google's computer. Great for **learning and data stories**.

A Python file (`.py`)

One plain text file, runs top to bottom on **your** computer. This is how apps and websites are built.

Editor (IDE)	Price	Good to know
PyCharm	free; Pro free for students	made for Python
VS Code	free	any language, many add-ons

IDE = Integrated Development Environment: editor + run button + debugger in one app.

Your first `.py` file

GOING FURTHER

1. Install Python and an IDE (guide on the course site)
2. New project, then a new file `hello.py`
3. Type your code
4. Click the green ▶ **Run**
5. The output appears at the bottom

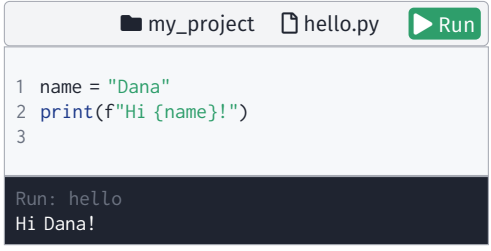


Illustration. PyCharm and VS Code look a bit different.

Your Colab code works here too, almost unchanged. Same Python!

The debugger: pause the program

GOING FURTHER

- Click next to a line number: a 🔴 **breakpoint**
- Run with the **Debug** button
- The program **stops before** that line

```
1  def total_price(prices):
2      for price in prices:
3          total = 0
4          total = total + price
5      return total
6
7  print(total_price([10, 20, 30]))
```

Illustration: the breakpoint on line 4; the paused line is yellow.

The debugger: pause the program

GOING FURTHER

- Click next to a line number: a 🔴 **breakpoint**
- Run with the **Debug** button
- The program **stops before** that line
- Now you see **every variable**, live. No `print` needed!

```
1  def total_price(prices):
2      for price in prices:
3          total = 0
4          total = total + price
5      return total
6
7  print(total_price([10, 20, 30]))
```

Illustration: the breakpoint on line 4; the paused line is yellow.

The bug from Lecture 8: `total` went back to 0 every round. Let's catch it live.

Step by step: watch the variables

GOING FURTHER

Press **Step Over** (PyCharm `F8`, VS Code `F10`) to run **one line**. The variables panel updates:

Round	`price`	`total`	**expected**
1	10	10	10

Step by step: watch the variables

GOING FURTHER

Press **Step Over** (PyCharm `F8`, VS Code `F10`) to run **one line**. The variables panel updates:

Round	`price`	`total`	**expected**
1	10	10	10
2	20	**20**	30
3	30	**30**	60

Step by step: watch the variables

GOING FURTHER

Press **Step Over** (PyCharm F8, VS Code F10) to run **one line**. The variables panel updates:

Round	`price`	`total`	**expected**
1	10	10	10
2	20	**20**	30
3	30	**30**	60

Caught live!

In round 2 `total` should be 30, but it is 20: line 3 set it back to 0.

Fix: move `total = 0` above the loop.

Step by step: watch the variables

GOING FURTHER

Press **Step Over** (PyCharm `F8`, VS Code `F10`) to run **one line**. The variables panel updates:

Round	`price`	`total`	**expected**
1	10	10	10
2	20	**20**	30
3	30	**30**	60

Caught live!

In round 2 `total` should be 30, but it is 20: line 3 set it back to 0.

Fix: move `total = 0` above the loop.

Same detective work as print debugging (L08), with a better magnifying glass.

`print` or the debugger?

GOING FURTHER

`print` debugging

- Works everywhere, also in Colab
- Quick: one line and run
- But you must guess *what* to print, and delete it later

The debugger

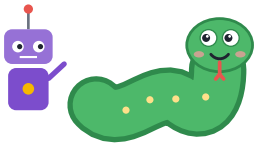
- See **all** variables at once
- Pause anywhere, step line by line
- Nothing to delete afterwards

Rule of thumb: a quick check → `print`. A puzzling bug → the debugger.

An AI pair programmer: Claude Code

GOING FURTHER

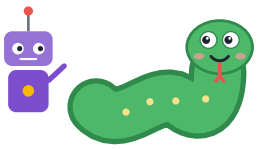
- Works **inside your project**: reads your files, writes and edits code, runs it, explains errors
- You ask in plain English: *“add a plot of sales per day to report.py”*
- It shows each change and asks before it acts



An AI pair programmer: Claude Code

GOING FURTHER

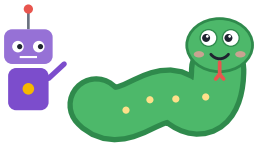
- Works **inside your project**: reads your files, writes and edits code, runs it, explains errors
- You ask in plain English: *“add a plot of sales per day to report.py”*
- It shows each change and asks before it acts
- **Live demo in class**



An AI pair programmer: Claude Code

GOING FURTHER

- Works **inside your project**: reads your files, writes and edits code, runs it, explains errors
- You ask in plain English: *“add a plot of sales per day to report.py”*
- It shows each change and asks before it acts
- **Live demo in class**



Claude Code needs a paid Claude plan. A free option for students: GitHub Copilot, free with GitHub Education. PyCharm and VS Code have AI helpers too.

The same rules still apply

GOING FURTHER

Before you use AI code

- **Explain every line** in your own words
- **Test** it with cases where you know the answer
- No personal data in prompts

A stronger tool doesn't change the rules from Lecture 8. In your homework and final project, you must be able to explain all of your code.



AI is a helper.
You are the programmer.

Topic 4

Wrap-up & your final project

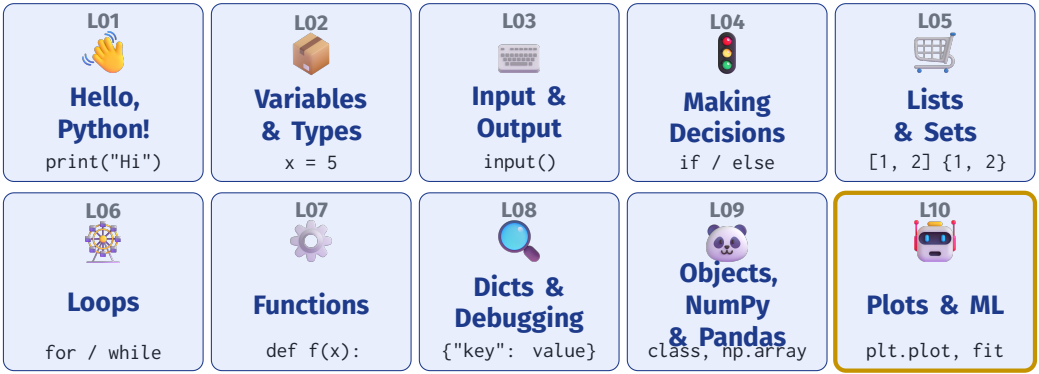
Lecture 10 • Plots & ML



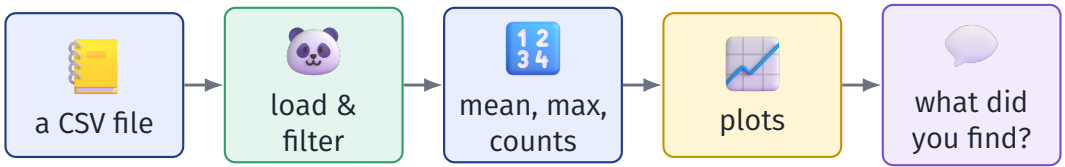
TODAY

1. Plots with Matplotlib
2. A first taste of machine learning
3. Code like a pro
4. **Wrap-up & your final project**

The whole course on one slide



Final project: your own data story



- Use **NumPy**, **Pandas** and **Matplotlib** together
- Plus your own functions, loops, decisions and a small class

A mini project, step 1: the numbers

L10_03_project_starter.ipynb

```
import pandas as pd
df = pd.read_csv("coffee_orders.csv")
days = ["Sun", "Mon", "Tue", "Wed", "Thu"]
totals = []
for day in days:
    one_day = df[df["day"] == day]
    totals.append(int(one_day["price"].sum()))
print(totals)
```

A mini project, step 1: the numbers

L10_03_project_starter.ipynb

```
import pandas as pd
df = pd.read_csv("coffee_orders.csv")
days = ["Sun", "Mon", "Tue", "Wed", "Thu"]
totals = []
for day in days:
    one_day = df[df["day"] == day]
    totals.append(int(one_day["price"].sum()))
print(totals)
```

OUTPUT

```
[108, 94, 121, 102, 113]
```

A mini project, step 1: the numbers

L10_03_project_starter.ipynb

```
import pandas as pd
df = pd.read_csv("coffee_orders.csv")
days = ["Sun", "Mon", "Tue", "Wed", "Thu"]
totals = []
for day in days:
    one_day = df[df["day"] == day]
    totals.append(int(one_day["price"].sum()))
print(totals)
```

OUTPUT

```
[108, 94, 121, 102, 113]
```

How much did the café earn each day?
A loop (L06), a list (L05), filtering (L09).

A mini project, step 2: the picture

Next cell, same notebook:

```
import matplotlib.pyplot as plt

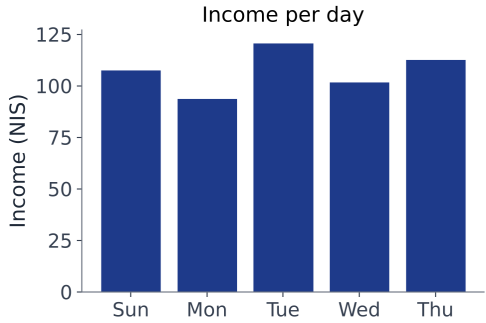
plt.bar(days, totals)
plt.title("Income per day")
plt.ylabel("Income (ILS)")
plt.show()
```

A mini project, step 2: the picture

Next cell, same notebook:

```
import matplotlib.pyplot as plt

plt.bar(days, totals)
plt.title("Income per day")
plt.ylabel("Income (ILS)")
plt.show()
```

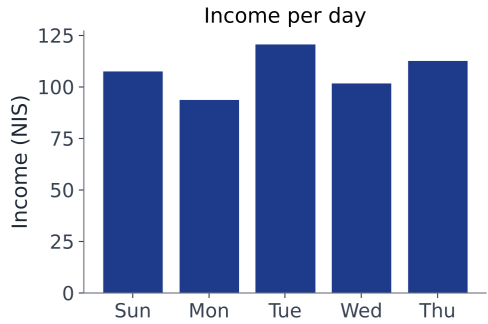


A mini project, step 2: the picture

Next cell, same notebook:

```
import matplotlib.pyplot as plt

plt.bar(days, totals)
plt.title("Income per day")
plt.ylabel("Income (ILS)")
plt.show()
```



Your story: Tuesday was the best day, Monday the slowest.

Final project: details and ideas

⚠ Requirements & deadline

- One CSV dataset you choose
- 3+ numbers (mean, max, counts), 2+ plots
- Your functions, a dict, a small class
- A 5-minute talk: what did you find?
- HW3: present in the **last week**

💡 Where to find data

- Your own data: spending, steps
- Public data: `data.gov.il`
- Practice datasets: `kaggle.com`

🎯 Pick a question you actually care about!

Bug Zoo: plotting bugs

BUG ZOO

```
days = [1, 2, 3, 4, 5, 6, 7]
sleep = [8, 7, 7, 6, 5, 4]
plt.plot(days, sleep)
```

ERROR

```
ValueError: x and y must have same
first dimension, but have shapes
(7,) and (6,)
```

Fix: one y value for every x value.

```
plt.xlable("Day")
```

ERROR

```
AttributeError: module
'matplotlib.pyplot' has no
attribute 'xlable'. Did you
mean: 'table'?
```

Fix: `plt.xlabel`. Even Python's guess can be wrong!

Bug Zoo: the 2-D surprise

BUG ZOO

```
from sklearn.linear_model import LinearRegression

hours = [1, 2, 3]
grades = [60, 70, 80]
model = LinearRegression()
model.fit(hours, grades)
```

Examples = a table

Even with one column.

Fix:

`[[1], [2], [3]]`

ERROR

```
ValueError: Expected 2D array, got 1D array instead:
```

(the first line of a longer error message)

AI Corner: polish your plots

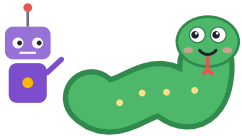
AI CORNER

A good prompt

“Here is my plot code. I am a beginner. Suggest 3 ways to make the plot clearer (title, labels, plot type) and explain each change.”

Double-check

Same numbers after the changes? Check one bar against your data.



Recap

✔ Today you learned

- Plots: line (time), bar (groups), scatter (pairs), histogram (spread)
- ML: create, `fit` on examples, `predict` new ones
- Pro tools: an IDE, a step-by-step debugger, AI (you still explain every line)
- Final project: CSV → Pandas → numbers → plot → story



What's next? You can keep going!

- Final project and exam: you are ready
- Free practice: Kaggle Learn, the official Python tutorial
- Use Python in *your* field: surveys, texts, sales, art



Thank you, and good luck!

3D emoji: Microsoft Fluent Emoji (MIT License). Pip the snake: original drawing. Full list: images/CREDITS.md